

4. Research and Teaching Setup

Rice Business 2026

Kerry Back

Rice Business

The Setup Tax

The old way

- Install the language, then the packages, then the editor
- Fight PATH variables, versions, and dependency errors
- Hours lost before a single line of research gets done

The new way

- Open Claude Code and say what you want to work in
- It installs the software, wires it up, and fixes what breaks
- You describe the environment; it builds it

This session is the research and teaching stack — Python, R, Stata, LaTeX, Quarto, Git, GitHub — and the single idea that one download can stand all of it up for you.

Academic Studio

One Download, Set Up for You

What it is

- A free, open-source desktop editor built for academics
- A customized build of **VS Codium** — the open-source core of VS Code, preconfigured
- Same editor, extensions, and workflow — with the academic toolchain already assembled
- One download from `academic-studio.com`, for macOS and Windows

What comes bundled

- **Claude Code** in a side panel, Quarto, Jupyter notebooks
- Python and R support, LaTeX (LaTeX Workshop), an Office/PDF viewer, spell-check
- A **Run Setup** wizard installs the supporting programs — Python, Node.js, Quarto, R, TinyTeX, Git, GitHub CLI
- A catalog of Recommended Plugins (Claude skills) to add

This is the fix for the setup tax. Instead of a checklist of installs — editor, extensions, languages, LaTeX, Git — you download one program and let its Run Setup wizard put the rest in place.

Academic Studio Is VS Code, Preconfigured

One Editor for Everything

What it is

- VS Code under the hood — one editor for every language you use
- Python, R, Stata, LaTeX, Markdown, Quarto — all in one window
- The extensions for each come preinstalled; Claude adds any others you need

Why it is the hub

- Claude lives in a side panel, aware of your open files
- Run code, edit papers, preview slides, commit to Git — without leaving
- Proposed edits appear as a **diff** you approve

Pick one home for your work. Academic Studio is that home — everything below happens inside it.

Notebooks, Right in the Editor

The screenshot displays a Jupyter notebook interface titled "my_notebook.ipynb". It contains three cells: a text cell, a code cell, and another text cell. The text cells are labeled "Text cell (Markdown)" and contain descriptive text about a tangency portfolio. The code cell is labeled "Code cell" and contains Python code for calculating optimal portfolio weights. The code cell also shows the output of the code, which is the optimal weights and Sharpe ratio.

Text cell (Markdown)

my_notebook.ipynb

Markdown

Tangency Portfolio

The tangency portfolio maximizes the Sharpe ratio: $(E[r] - r_f) / \sigma$

Code cell

```
import numpy as np
from scipy.optimize import minimize

weights = minimize(neg_sharpe, x0, ...)
```

Output:

Optimal weights: [0.35, 0.25, 0.40] Sharpe ratio: 1.42

Markdown

The optimal portfolio allocates 35% to stocks, 25% to bonds, and 40% to real estate.

A Jupyter notebook interleaves code, output, and prose — ideal for analysis you want to narrate. Academic

Languages

Python and R

Python

- The default for data work, machine learning, and apps
- Claude installs libraries for you — `pip install pandas`
- On a Mac the command is `python3`; Claude adjusts automatically

R

- The statistician's language — and much of empirical social science
- Claude writes and runs `.R` scripts, and knows the tidyverse and common packages
- Same loop: describe the analysis, it writes the code and runs it

You do not have to choose a language on ideological grounds. Claude is fluent in both — work in what your field and your co-authors use.

Stata

Claude writes your do-files

- Describe the regression, the panel setup, the table — Claude writes the `.do` file
- It knows Stata syntax, `esttab`, `reghdfe`, and the usual toolkit
- Run it interactively, or let Claude run it in batch and read the log

The workflow

- Keep your data and do-files in a project folder
- “Add clustered standard errors and re-run” — it edits the do-file and executes
- The log becomes something Claude reads back and explains

Stata’s GUI is not the point of contact. The `.do` file is — and that is exactly the kind of text Claude writes, edits, and reruns well.

Writing

LaTeX

For papers that look right

- The standard for math-heavy papers, journals, and job-market work
- Claude drafts sections, builds tables, and fixes the errors LaTeX is famous for
- It reads the cryptic compile errors and resolves them — the worst part, gone

Tables from results

- Point it at your regression output; it produces the formatted LaTeX table
- Change a spec, regenerate the table — no hand-editing columns
- Figures, citations, and BibTeX handled the same way

The tedium of LaTeX — the debugging, the table formatting, the reference wrangling — is precisely what AI removes.

Quarto: One Source, Many Outputs

Write once

Plain-text `.qmd` files:
prose, code, and math
together

Render anywhere

The same source
becomes a paper
(PDF), a website, or
slides

These slides

This deck is Quarto —
rendered to reveal.js,
exportable to PDF or
PowerPoint

Quarto is the modern successor to R Markdown and works with Python, R, and Stata alike. Because it is plain text, Claude authors and restyles whole documents — as it did for every deck in this course.

Git & GitHub

Git: Track Changes for a Whole Project

The idea

- Git is **Track Changes** for an entire folder of files
- Every save you choose is a **commit** — a snapshot you can return to
- Break something? Roll back to any past version

With Claude, it is invisible

- Claude Code installs Git and commits for you
- "Commit this," "show me what changed," "go back to yesterday's version"
- The safety net that lets you say yes to bigger changes

The reason you can let AI edit your project freely is that Git remembers every state — nothing is ever truly lost.

GitHub: Backup, Sharing, Publishing

Your code in the cloud

- GitHub hosts your Git projects — backed up and shareable
- “Push this project to a new GitHub repo” — Claude sets up the CLI and does it
- Co-authors clone it; you track issues and versions together

GitHub Pages: free hosting

- Name a page `index.html`, push, flip on Pages
- Live at a `github.io` address — no server, no cost
- Perfect for a course site, a paper’s companion page, or a small app

For solo research, GitHub is mainly backup plus a launch pad. For teaching, it is free hosting for anything you want students to reach.

The One-Download Setup

The easy path

- Download Academic Studio and open it once
- Click **Run Setup** — it installs Python, Node, Quarto, R, TinyTeX, Git, and the GitHub CLI
- Add Recommended Plugins from the catalog, and you are ready to work

Or just ask Claude

- Already inside, Claude Code can stand up a project from one paragraph:
- "Set up a research project here: Python with pandas, numpy, statsmodels, matplotlib; R; Quarto and LaTeX. Make folders for data, code, and output. Initialize Git and push to a new private GitHub repo."
- It installs each piece, verifies it, and hands back a working, version-controlled project

One download, or one paragraph. Either way the setup tax is paid once, by the machine — not by you, by hand.

The Point

The environment is no longer something you assemble by hand and pray works. You **describe the stack you want** — languages, writing tools, version control — and Claude Code builds it, fixes it, and keeps it working. Your time goes to the research, not the setup.

Breakout

Questions to Discuss

- What is in your current research stack — and what part of setting it up costs you the most time?
- Which language do you and your co-authors actually work in? Where would AI help across that boundary?
- Do you use version control now? What has “losing work” cost you that Git would have prevented?
- What would you want on a free GitHub Pages site for one of your courses?